

Software Requirements Specification (SRS)

for

Physics Sphere



Indian Institute of Information Technology, Allahabad

Prepared by:

IIT2023132 (Ishita)
IIT2023140 (Nitya Bhavsar)
IIT2023143 (Kartik Sharma)
IIT2023144 (Ayushi Songara)
IIT2023147 (Priyam J Chakrabarthi)
IIT2023149 (Yash Kamthan)

Date: April 30, 2025

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Document Conventions	1
1.3	Intended Audience and Reading Suggestions	1
1.4	Product Scope	1
1.5	References	2
2	Overall Description	2
2.1	Product Perspective	2
2.2	Product Functions	2
2.3	User Classes and Characteristics	2
2.4	Operating Environment	3
2.5	Design and Implementation Constraints	3
2.6	User Documentation	3
2.7	Assumptions and Dependencies	3
3	External Interface Requirements	4
3.1	User Interfaces	4
3.2	Hardware Interfaces	4
3.3	Software Interfaces	4
3.4	Communications Interfaces	4
4	System Features	5
4.1	General Simulation Flow	5
4.1.1	Description and Priority	5
4.1.2	Stimulus/Response Sequences	5
4.1.3	Functional Requirements	5
4.2	Projectile Motion	6
4.2.1	Description and Priority	6
4.2.2	Functional Requirements	6
4.3	Relative Motion	6
4.3.1	Description and Priority	6
4.3.2	Functional Requirements	6
4.4	Circular Motion	6
4.4.1	Description and Priority	6
4.4.2	Functional Requirements	6
4.5	Pendulum Motion	7
4.5.1	Description and Priority	7
4.5.2	Functional Requirements	7
4.6	Spring Motion	7
4.6.1	Description and Priority	7
4.6.2	Functional Requirements	7
4.7	Buoyancy Simulation	7
4.7.1	Description and Priority	7
4.7.2	Functional Requirements	7
5	Other Nonfunctional Requirements	8

5.1	Performance Requirements	8
5.2	Safety Requirements	8
5.3	Security Requirements	8
5.4	Software Quality Attributes	8
5.5	Business Rules	9
6	Other Requirements	9
A	Glossary	10
B	Analysis Models	11
C	To Be Determined List	11

1. Introduction

1.1 Purpose

The purpose of this project is to develop an interactive web-based Physics Concept Visualizer using Three.js, a JavaScript 3D rendering library. This tool is designed to help users, especially students and educators, understand and explore physics concepts like projectile motion and wave mechanics through real-time 3D simulations. This SRS outlines the functional and non-functional requirements for version 1.0 of the application.

1.2 Document Conventions

- **REQ-#** is used to represent uniquely numbered software requirements.
- Important terms such as *simulation*, *waveform*, and *initial velocity* are italicized when first introduced.
- GUI element names (e.g., **Start**, **Reset**) are shown in bold.
- All simulation features are organized by system function in Section 4.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- **Developers:** To implement the described functionality and structure.
- **Testers:** To verify compliance with specified behavior.
- **Educators/Students:** To understand what the software provides.
- **Project Managers:** To monitor feature completion.

x‘ It is recommended to begin with Section 2 for an overview, then refer to Sections 3 and 4 for detailed interface and functional requirements.

1.4 Product Scope

The **Physics Concept Visualizer** aims to improve understanding of classical physics concepts by providing dynamic, interactive 3D simulations that users can control via input parameters. The tool supports real-time manipulation of inputs like angle and velocity to visualize motion and waves, making abstract theory more tangible and intuitive.

1.5 References

- Three.js documentation: <https://threejs.org/docs/>
- WebGL Specification: <https://www.khronos.org/webgl/>
- Hewitt, Paul G., *Conceptual Physics*
- Project GitHub Repository: <https://github.com/PriyamJChakrabarty/PhysicsConceptVisualizer>
git

2. Overall Description

2.1 Product Perspective

The **Physics Concept Visualizer** is a standalone, client-side web application built using **Three.js** and JavaScript. It is not part of a larger system but may be integrated into e-learning platforms or embedded into websites as an interactive module. The system leverages the WebGL API via Three.js to render 3D visualizations directly in the browser, requiring no backend or server-side processing.

2.2 Product Functions

At a high level, the system provides the following functionality:

- Simulate projectile motion with adjustable initial velocity and launch angle.
- Visualize wave propagation using sine and cosine functions.
- Provide a 3D viewport with camera controls for orbiting, zooming, and panning.
- Start, pause, and reset simulations.
- Real-time updates based on user input.

2.3 User Classes and Characteristics

- **Students:** Users with minimal technical knowledge, primarily using the tool for learning physics through visual examples.
- **Educators:** May use the tool in classrooms or demonstrations to aid in teaching.

- **Developers:** For extending, debugging, or maintaining the project. Familiarity with JavaScript and Three.js is expected.

2.4 Operating Environment

- Modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge (latest versions).
- Desktop or laptop computers with WebGL support and GPU acceleration.
- Internet connection not required after initial load; runs fully client-side.

2.5 Design and Implementation Constraints

- Application must use **Three.js** for all 3D rendering tasks.
- The interface must be responsive and maintain functionality across screen sizes.
- All simulations must run in real time within the browser.
- Minimal external libraries to reduce load time and maintain simplicity.

2.6 User Documentation

- README file with setup and usage instructions.
- In-app tooltips and labels for input fields and controls.

2.7 Assumptions and Dependencies

- Users are assumed to have JavaScript enabled in their browsers.
- Assumes basic physics knowledge to understand visualizations.
- Assumes the system will be deployed on environments with WebGL support.
- No dependency on backend APIs or server-side technologies.

3. External Interface Requirements

3.1 User Interfaces

The application provides a graphical user interface (GUI) that includes:

- A 3D rendering canvas powered by Three.js for simulation visualization.
- Input controls such as sliders, number fields, and buttons to adjust simulation parameters (e.g., velocity, angle, frequency).
- Buttons for **Start**, **Pause**, and **Reset** actions.
- Real-time feedback and updates as inputs are modified.
- Tooltip hints and labels to improve usability.
- Mouse-based interaction for rotating, zooming, and panning the 3D scene.

3.2 Hardware Interfaces

- No specialized hardware is required.
- Compatible with standard desktop or laptop systems equipped with GPU-accelerated graphics rendering.
- Requires a keyboard and mouse (or touchpad) for user input and interaction.

3.3 Software Interfaces

- Built using **Three.js** for WebGL-based 3D rendering.
- Developed in **JavaScript**, **HTML5**, and **CSS3**.
- May optionally use browser APIs (e.g., `requestAnimationFrame`, DOM manipulation).
- No server-side components or databases are used.

3.4 Communications Interfaces

- The application operates entirely client-side and does not require communication with a backend server.

- If embedded or extended, may use standard web protocols (e.g., HTTP/HTTPS) for integration into learning management systems.
- All communication (if any in the future) shall use secure HTTPS protocol and follow standard browser security models.

4. System Features

4.1 General Simulation Flow

4.1.1 Description and Priority

This defines the overall user journey: launching the app, selecting a simulation mode, inputting parameters, running the animation, and having the ability to pause/reset or restart with adjustments. **Priority: High**

4.1.2 Stimulus/Response Sequences

- User opens the application → 3D canvas loads.
- User selects a simulation mode (e.g., Projectile or Buoyancy).
- Parameters are entered or adjusted.
- User starts the simulation → system animates motion.
- User can pause or reset the simulation, and optionally adjust inputs to restart.

4.1.3 Functional Requirements

- REQ-1: The system shall display a 3D canvas on launch.
- REQ-2: The user shall be able to choose one of the six physics modes.
- REQ-3: The system shall accept relevant parameters per selected mode.
- REQ-4: The simulation shall be startable, pausable, and resettable.
- REQ-5: The user shall be able to modify inputs and restart the simulation.

4.2 Projectile Motion

4.2.1 Description and Priority

Simulates parabolic motion under the influence of gravity with configurable velocity and angle. **Priority: High**

4.2.2 Functional Requirements

- REQ-6: The user shall input angle and velocity.
- REQ-7: The system shall display a 3D animation of projectile trajectory.

4.3 Relative Motion

4.3.1 Description and Priority

Visualizes the effect of relative reference frames (e.g., boat in river, train with observer). **Priority: Medium**

4.3.2 Functional Requirements

- REQ-8: The user shall input frame velocity and object velocity.
- REQ-9: The system shall render combined motion with clear reference visualization.

4.4 Circular Motion

4.4.1 Description and Priority

Demonstrates angular velocity, centripetal force, and Circular motion. **Priority: Medium**

4.4.2 Functional Requirements

- REQ-10: The user shall input radius and angular speed.
- REQ-11: The object shall follow a Circular path in the 3D view.

4.5 Pendulum Motion

4.5.1 Description and Priority

Simulates simple harmonic motion of a pendulum based on gravity and string length. **Priority: High**

4.5.2 Functional Requirements

- REQ-12: The user shall input length and initial angle.
- REQ-13: The pendulum shall animate using calculated period and damping (if any).

4.6 Spring Motion

4.6.1 Description and Priority

Models Hooke's law to demonstrate motion of a mass-spring system. **Priority: Medium**

4.6.2 Functional Requirements

- REQ-14: The user shall input spring constant and mass.
- REQ-15: The animation shall show vertical or horizontal oscillation.

4.7 Buoyancy Simulation

4.7.1 Description and Priority

Visualizes buoyant forces and floating/sinking behavior based on Archimedes' principle. **Priority: Medium**

4.7.2 Functional Requirements

- REQ-16: The user shall input object volume and fluid density.
- REQ-17: The simulation shall animate object floating or sinking based on net force.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The application must render simulations at a minimum of 30 frames per second (FPS) for smooth animation.
- Input changes (e.g., velocity, angle) must be reflected in the visualization within 200 milliseconds.
- The system should handle at least 10 simultaneous simulations without degradation in performance.
- The browser window should not freeze or crash when the simulation is running or paused.

5.2 Safety Requirements

- The application is designed to be safe for use in classroom and educational settings.
- No safety risks are associated with this application as it operates entirely within the user's browser.
- The software does not interact with any physical devices or external hardware that may cause harm.

5.3 Security Requirements

- The application does not handle sensitive data or require user authentication, therefore no personal information is stored.
- If any future communication with external systems is implemented, all data exchanges will use HTTPS to ensure security.
- The system must be resilient to common security vulnerabilities such as Cross-Site Scripting (XSS) or Cross-Site Request Forgery (CSRF).

5.4 Software Quality Attributes

- **Usability:** The application must be user-friendly and intuitive, requiring no prior technical knowledge. Simple instructions will be provided for first-time users.

- **Maintainability:** The codebase should be modular and well-documented, making future updates and bug fixes easier to implement.
- **Portability:** The application must work across different browsers and devices, including desktop and tablet platforms, with minimal adjustments needed.
- **Reliability:** The software should operate continuously without unexpected crashes or errors under normal usage.
- **Testability:** The system should include sufficient testing coverage, including unit tests for major features and integration tests for different components.

5.5 Business Rules

- Only users with an internet connection and a compatible web browser (Chrome, Firefox, Edge) can use the system.
- The application should be available for free, and any future versions may include optional premium features.
- The tool may be licensed for educational use by schools, colleges, and online learning platforms.

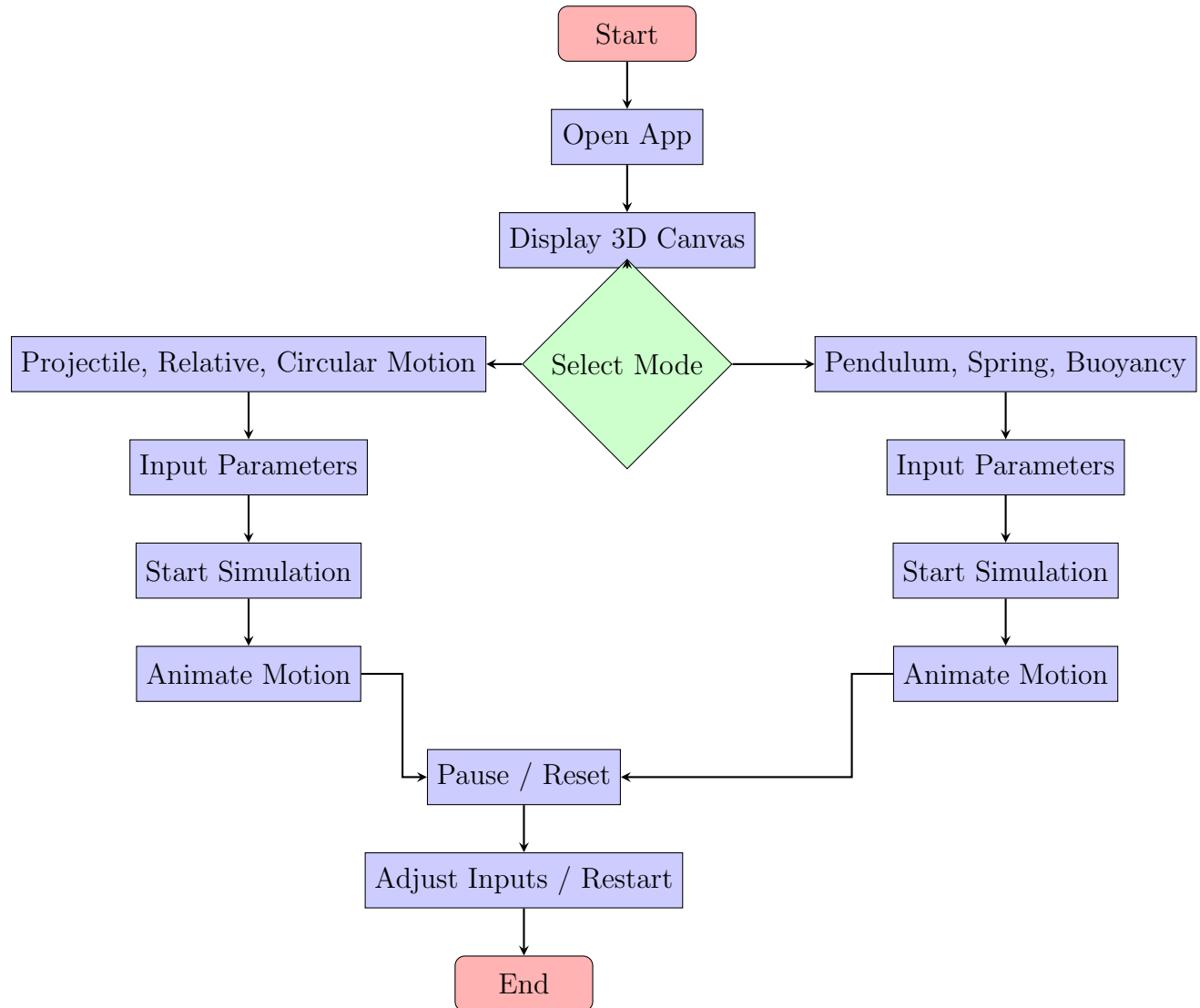
6. Other Requirements

- The application must support dynamic window resizing, ensuring the interface adapts seamlessly to different screen sizes.
- Future versions may include the ability to export simulation data (e.g., trajectories, waveforms) for use in reports or further analysis.
- The system should be capable of being extended to support additional physical concepts (e.g., Circular motion, electrical circuits).
- The tool should allow users to export screenshots of the simulation for documentation or educational purposes.
- The system should be internationalization-ready, supporting multiple languages in future updates.

A. Glossary

- **Three.js:** A powerful JavaScript library that simplifies the creation and rendering of 3D graphics in a web browser using WebGL.
- **WebGL:** Web Graphics Library, a browser-supported JavaScript API for rendering high-performance interactive 3D and 2D graphics without plug-ins.
- **Projectile Motion:** A type of motion experienced by an object thrown into the air, subject to only gravitational acceleration.
- **Relative Motion:** Motion of an object as observed from a particular frame of reference, which itself may be in motion.
- **Circular Motion:** The movement of an object in a Circular path around a central axis, typically described using angular displacement and velocity.
- **Pendulum Motion:** Oscillatory motion exhibited by a suspended mass swinging under the influence of gravity.
- **Spring Motion:** Motion governed by Hooke's law, where a mass attached to a spring moves in harmonic oscillation based on stiffness and displacement.
- **Buoyancy:** The upward force exerted by a fluid on a submerged or partially submerged object, determining whether it floats or sinks.
- **Simulation:** A virtual representation of real-world physics phenomena executed by a program to help visualize and understand the behavior dynamically.
- **Amplitude:** The maximum extent of a vibration or oscillation, measured from the position of equilibrium.
- **Frequency:** The number of complete oscillations or cycles per unit time.
- **Control Panel:** The user interface section containing buttons for starting, pausing, resetting, and stepping through simulations.
- **Camera Controls:** Interactive tools allowing users to pan, zoom, and orbit the 3D scene to view simulations from various angles.
- **Visualization Modes:** Options to toggle different graphical representations, such as basic trajectory or enhanced views with vectors and additional effects.
- **Parameter Inputs:** Input fields or sliders through which users enter simulation-specific values like angle, velocity, or mass.

B. Analysis Models



C. To Be Determined List

1. TBD-1: Persist user preferences (e.g., last-used parameters) across sessions.
2. TBD-2: Implement mobile-friendly touch controls for camera and parameter adjustments.
3. TBD-3: Provide multi-language support for UI text and labels.